

```

import warnings
warnings.filterwarnings("ignore")

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

FILE = "/content/superstore.csv.csv"

try:
    df = pd.read_csv(FILE, encoding="latin1")
except FileNotFoundError:
    print("ERROR: superstore.csv not found.")
    print("Put superstore.csv in the same folder as this Python file.")
    exit()

print("\n" + "=" * 60)
print("DATASET LOADED SUCCESSFULLY")
print("=" * 60)

print("\nFirst 5 rows:")
print(df.head())

print("\nDataset Shape:")
print(df.shape)

print("\nColumn Names:")
print(df.columns.tolist())

print("\nDataset Information:")
df.info()

print("\nMissing Values:")
print(df.isnull().sum())

print("\nDuplicate Rows:")
print(df.duplicated().sum())

df.columns = df.columns.str.strip()

df = df.drop_duplicates()
numeric_columns = ["Sales", "Profit", "Quantity", "Discount"]

for col in numeric_columns:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors="coerce")

for col in df.select_dtypes(include=np.number).columns:
    df[col] = df[col].fillna(df[col].median())

for col in df.select_dtypes(include="object").columns:
    if df[col].isnull().sum() > 0:
        df[col] = df[col].fillna(df[col].mode()[0])

print("\nMissing values after preprocessing:")
print(df.isnull().sum())

print("\n" + "=" * 60)
print("DESCRIPTIVE STATISTICS")
print("=" * 60)

print(df.describe())

total_sales = df["Sales"].sum()
total_profit = df["Profit"].sum()

```

```

total_quantity = df["Quantity"].sum()

if "Order ID" in df.columns:
    total_orders = df["Order ID"].nunique()
else:
    total_orders = len(df)

profit_margin = (total_profit / total_sales) * 100

print("\n" + "=" * 60)
print("KEY PERFORMANCE INDICATORS")
print("=" * 60)

print(f"Total Sales      : ${total_sales:,.2f}")
print(f"Total Profit       : ${total_profit:,.2f}")
print(f"Total Quantity      : {total_quantity:,.0f}")
print(f"Total Orders        : {total_orders:,}")
print(f"Profit Margin        : {profit_margin:.2f}%")

category_sales = (
    df.groupby("Category")["Sales"]
    .sum()
    .sort_values(ascending=False)
)

print("\nSales by Category:")
print(category_sales)

plt.figure(figsize=(8, 5))
sns.barplot(
    x=category_sales.index,
    y=category_sales.values
)

plt.title("Sales by Category")
plt.xlabel("Category")
plt.ylabel("Sales")
plt.tight_layout()
plt.show()

category_profit = (
    df.groupby("Category")["Profit"]
    .sum()
    .sort_values(ascending=False)
)

print("\nProfit by Category:")
print(category_profit)

plt.figure(figsize=(8, 5))
sns.barplot(
    x=category_profit.index,
    y=category_profit.values
)

plt.title("Profit by Category")
plt.xlabel("Category")
plt.ylabel("Profit")
plt.tight_layout()
plt.show()

region_sales = (
    df.groupby("Region")["Sales"]
    .sum()
    .sort_values(ascending=False)
)

print("\nSales by Region:")
print(region_sales)

plt.figure(figsize=(9, 5))
sns.barplot(
    x=region_sales.index,
    y=region_sales.values
)

plt.title("Sales by Region")
plt.xlabel("Region")

```

```

plt.ylabel("Sales")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()

region_profit = (
    df.groupby("Region")["Profit"]
    .sum()
    .sort_values(ascending=False)
)

print("\nProfit by Region:")
print(region_profit)

plt.figure(figsize=(9, 5))
sns.barplot(
    x=region_profit.index,
    y=region_profit.values
)

plt.title("Profit by Region")
plt.xlabel("Region")
plt.ylabel("Profit")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()

subcategory_sales = (
    df.groupby("Sub-Category")["Sales"]
    .sum()
    .sort_values(ascending=False)
)

print("\nTop Sub-Categories by Sales:")
print(subcategory_sales.head(10))

plt.figure(figsize=(10, 6))

subcategory_sales.head(10).sort_values().plot(
    kind="barh"
)

plt.title("Top 10 Sub-Categories by Sales")
plt.xlabel("Sales")
plt.ylabel("Sub-Category")
plt.tight_layout()
plt.show()

numeric_df = df.select_dtypes(include=np.number)

correlation = numeric_df.corr()

print("\n" + "=" * 60)
print("CORRELATION MATRIX")
print("=" * 60)

print(correlation)

plt.figure(figsize=(10, 7))

sns.heatmap(
    correlation,
    annot=True,
    fmt=".2f",
    cmap="coolwarm"
)

plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()

plt.figure(figsize=(9, 6))

sns.scatterplot(
    data=df,
    x="Sales",
    y="Profit",

```

```

        hue="Category",
        alpha=0.6
    )

plt.title("Sales vs Profit")
plt.xlabel("Sales")
plt.ylabel("Profit")
plt.tight_layout()
plt.show()

plt.figure(figsize=(9, 5))

sns.histplot(
    df["Sales"],
    bins=30,
    kde=True
)

plt.title("Sales Distribution")
plt.xlabel("Sales")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

plt.figure(figsize=(9, 5))

sns.histplot(
    df["Profit"],
    bins=30,
    kde=True
)

plt.title("Profit Distribution")
plt.xlabel("Profit")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

print("\n" + "=" * 60)
print("MACHINE LEARNING - SALES PREDICTION")
print("=" * 60)

features = [
    "Quantity",
    "Discount",
    "Profit"
]

categorical_features = [
    "Category",
    "Sub-Category",
    "Region"
]

features = [
    col for col in features
    if col in df.columns
]

categorical_features = [
    col for col in categorical_features
    if col in df.columns
]

X = df[features + categorical_features]
y = df["Sales"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("Training samples:", len(X_train))
print("Testing samples :", len(X_test))

```

```

preprocessor = ColumnTransformer(
    transformers=[
        (
            "categorical",
            OneHotEncoder(
                handle_unknown="ignore"
            ),
            categorical_features
        )
    ],
    remainder="passthrough"
)

model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("regressor", LinearRegression())
    ]
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)

rmse = np.sqrt(
    mean_squared_error(y_test, y_pred)
)

r2 = r2_score(y_test, y_pred)

print("\nMODEL PERFORMANCE")
print("-" * 40)

print(f"MAE : {mae:.2f}")
print(f"RMSE : {rmse:.2f}")
print(f"R² : {r2:.4f}")

results = pd.DataFrame({
    "Actual Sales": y_test.values,
    "Predicted Sales": y_pred
})

print("\nActual vs Predicted Sales:")
print(results.head(10))

plt.figure(figsize=(9, 6))

plt.scatter(
    y_test,
    y_pred,
    alpha=0.6
)

plt.xlabel("Actual Sales")
plt.ylabel("Predicted Sales")
plt.title("Actual vs Predicted Sales")

plt.tight_layout()
plt.show()

best_sales_category = category_sales.idxmax()
best_profit_category = category_profit.idxmax()
best_sales_region = region_sales.idxmax()
best_profit_region = region_profit.idxmax()

print("\n" + "=" * 60)
print("BUSINESS INSIGHTS")
print("=" * 60)

print(
    f"\n1. Highest sales category: "

```

```

    f"{best_sales_category}"
)

print(
    f"2. Highest profit category: "
    f"{best_profit_category}"
)

print(
    f"3. Highest sales region: "
    f"{best_sales_region}"
)

print(
    f"4. Highest profit region: "
    f"{best_profit_region}"
)

print(
    f"5. Overall profit margin: "
    f"{profit_margin:.2f}%"
)

print(
    f"6. The predictive model achieved an R2 score of "
    f"{r2:.4f}."
)

print("\n" + "=" * 60)
print("BUSINESS RECOMMENDATIONS")
print("=" * 60)

print("""
1. Monitor high-performing categories and regions.

2. Investigate categories with comparatively low profit.

3. Review discount strategies where discounts may reduce
   profitability.

4. Track Sales, Profit and Profit Margin as regular KPIs.

5. Use EDA visualizations and dashboards for continuous
   business monitoring.

6. Improve the predictive model by adding more relevant
   customer, product and time-related features.

7. Compare multiple machine learning algorithms in future
   work and select the model based on appropriate evaluation
   metrics.
""")

print("\n" + "=" * 60)
print("END-TO-END ANALYTICS PROJECT COMPLETED")
print("=" * 60)

print("""
Workflow completed:

✓ Data collection
✓ Data preprocessing
✓ Missing-value handling
✓ Duplicate handling
✓ Descriptive statistics
✓ EDA
✓ Visualization
✓ KPI calculation
✓ Correlation analysis
✓ Machine learning
✓ Sales prediction
✓ Model evaluation
✓ Business insights
✓ Recommendations
""")

```



```
=====
DATASET LOADED SUCCESSFULLY
=====
```

First 5 rows:

	i»¿Row ID	Order ID	Order Date	Ship Date	Ship Mode	\
0	40098	CA-2014-AB10015140-41954	11/11/2014	11/13/2014	First Class	
1	26341	IN-2014-JR162107-41675	2/5/2014	2/7/2014	Second Class	
2	25330	IN-2014-CR127307-41929	10/17/2014	10/18/2014	First Class	
3	13524	ES-2014-KM1637548-41667	1/28/2014	1/30/2014	First Class	
4	47221	SG-2014-RH9495111-41948	11/5/2014	11/6/2014	Same Day	

	Customer ID	Customer Name	Segment	Postal Code	City	\
0	AB-100151402	Aaron Bergman	Consumer	73120.0	Oklahoma City	
1	JR-162107	Justin Ritter	Corporate	NaN	Wollongong	
2	CR-127307	Craig Reiter	Consumer	NaN	Brisbane	
3	KM-1637548	Katherine Murray	Home Office	NaN	Berlin	
4	RH-9495111	Rick Hansen	Consumer	NaN	Dakar	

	...	Product ID	Category	Sub-Category	\
0	...	TEC-PH-5816	Technology	Phones	
1	...	FUR-CH-5379	Furniture	Chairs	
2	...	TEC-PH-5356	Technology	Phones	
3	...	TEC-PH-5267	Technology	Phones	
4	...	TEC-CO-6011	Technology	Copiers	

		Product Name	Sales	Quantity	Discount	\
0		Samsung Convoy 3	221.98	2	0.0	
1	Novimex Executive Leather Armchair, Black		3709.40	9	0.1	
2	Nokia Smart Phone, with Caller ID		5175.17	9	0.1	
3	Motorola Smart Phone, Cordless		2892.51	5	0.1	
4	Sharp Wireless Fax, High-Speed		2832.96	8	0.0	

	Profit	Shipping Cost	Order Priority
0	62.15	40.77	High
1	-288.77	923.63	Critical
2	919.97	915.49	Medium
3	-96.54	910.16	Medium
4	311.52	903.04	Critical

[5 rows x 24 columns]

Dataset Shape:
(1000, 24)

Column Names:

['i»¿Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode', 'Customer ID', 'Customer Name', 'Segment', 'Postal Code', 'C

Dataset Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1000 entries, 0 to 999

Data columns (total 24 columns):

#	Column	Non-Null Count	Dtype
0	i»¿Row ID	1000 non-null	int64
1	Order ID	1000 non-null	object
2	Order Date	1000 non-null	object
3	Ship Date	1000 non-null	object
4	Ship Mode	1000 non-null	object
5	Customer ID	1000 non-null	object
6	Customer Name	1000 non-null	object
7	Segment	1000 non-null	object
8	Postal Code	194 non-null	float64
9	City	1000 non-null	object
10	State	1000 non-null	object
11	Country	1000 non-null	object
12	Region	1000 non-null	object
13	Market	1000 non-null	object
14	Product ID	1000 non-null	object
15	Category	1000 non-null	object
16	Sub-Category	1000 non-null	object
17	Product Name	1000 non-null	object
18	Sales	1000 non-null	float64
19	Quantity	1000 non-null	int64
20	Discount	1000 non-null	float64
21	Profit	1000 non-null	float64
22	Shipping Cost	1000 non-null	float64
23	Order Priority	1000 non-null	object

dtypes: float64(5), int64(2), object(17)

memory usage: 187.6+ KB

Missing Values:

i»¿Row ID 0

```
Order ID      0
Order Date    0
Ship Date     0
Ship Mode     0
Customer ID   0
Customer Name 0
Segment       0
Postal Code   806
City          0
State         0
Country       0
Region        0
Market        0
Product ID    0
Category      0
Sub-Category  0
Product Name  0
Sales         0
Quantity      0
Discount      0
Profit        0
Shipping Cost 0
Order Priority 0
dtype: int64
```

Duplicate Rows:
0

Missing values after preprocessing:

```
i»¿Row ID      0
Order ID       0
Order Date     0
Ship Date      0
Ship Mode      0
Customer ID    0
Customer Name  0
Segment        0
Postal Code    0
City           0
State          0
Country        0
Region         0
Market         0
Product ID     0
Category       0
Sub-Category   0
Product Name   0
Sales          0
Quantity       0
Discount       0
Profit         0
Shipping Cost  0
Order Priority  0
dtype: int64
```

=====

DESCRIPTIVE STATISTICS

=====

	i»¿Row ID	Postal Code	Sales	Quantity	Discount \
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000
mean	25079.328000	59284.021000	1710.971470	5.55800	0.092840
std	12897.726632	15055.499595	1259.239238	2.71846	0.148666
min	58.000000	2920.000000	1.910000	1.00000	0.000000
25%	15118.750000	60564.000000	826.907500	4.00000	0.000000
50%	25084.500000	60564.000000	1585.115000	5.00000	0.000000
75%	34524.000000	60564.000000	2477.812500	7.00000	0.150000
max	51284.000000	98198.000000	9892.740000	14.00000	0.800000

	Profit	Shipping Cost
count	1000.000000	1000.000000
mean	288.920440	272.384897
std	574.504782	176.160716
min	-3059.820000	1.070000
25%	10.037500	209.827500
50%	190.685000	258.897500
75%	518.872500	351.070250
max	4946.370000	923.630000

=====

KEY PERFORMANCE INDICATORS

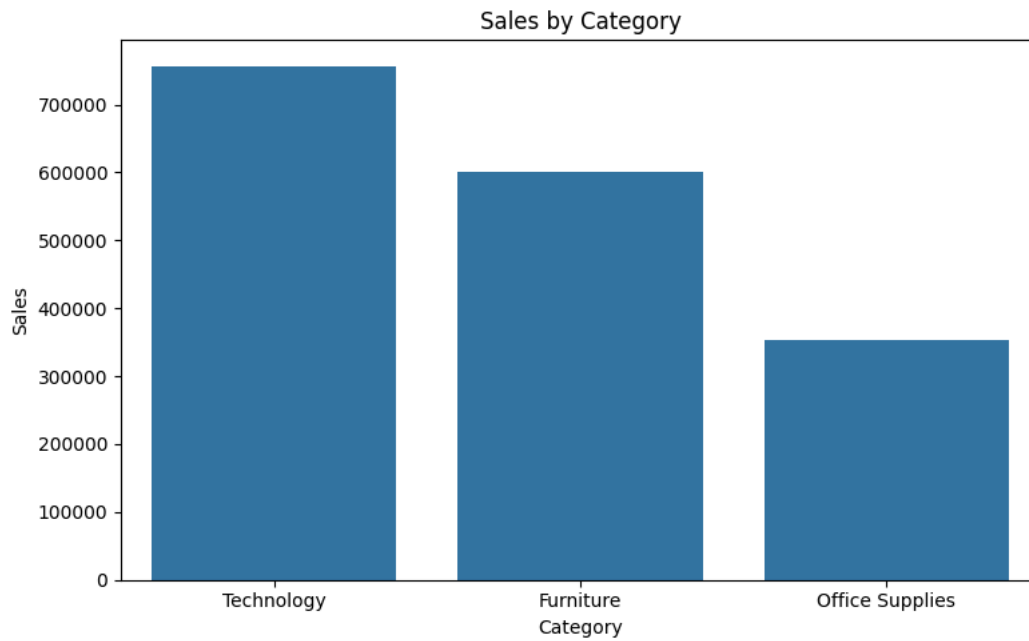
=====

Total Sales : \$1,710,971.47
Total Profit : \$288,920.44
Total Quantity : 5,558
Total Orders : 867

Profit Margin : 16.89%

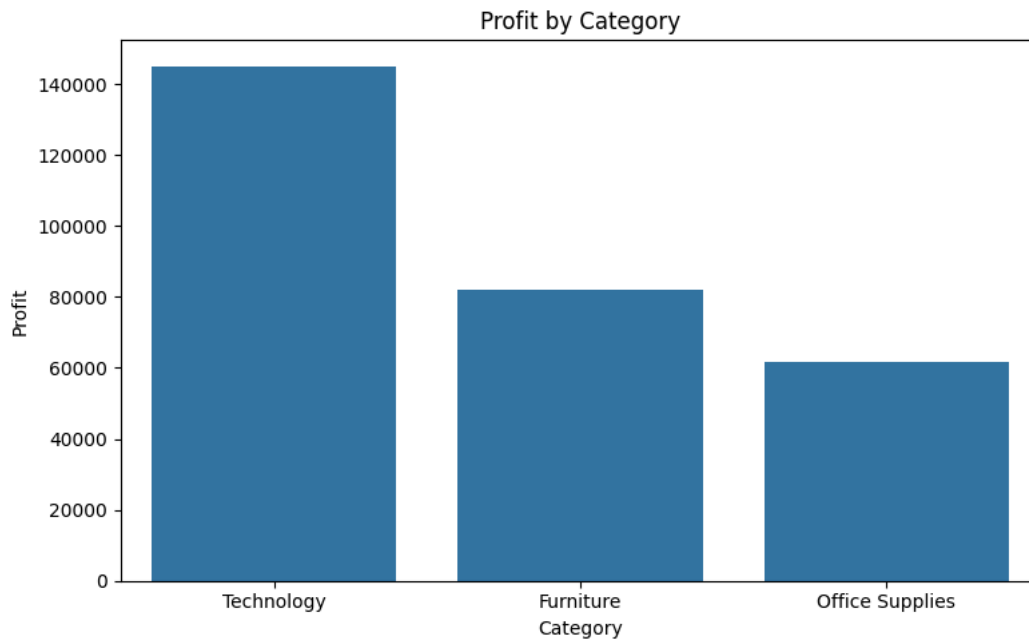
Sales by Category:

```
Category
Technology      756983.02
Furniture        601492.31
Office Supplies  352496.14
Name: Sales, dtype: float64
```



Profit by Category:

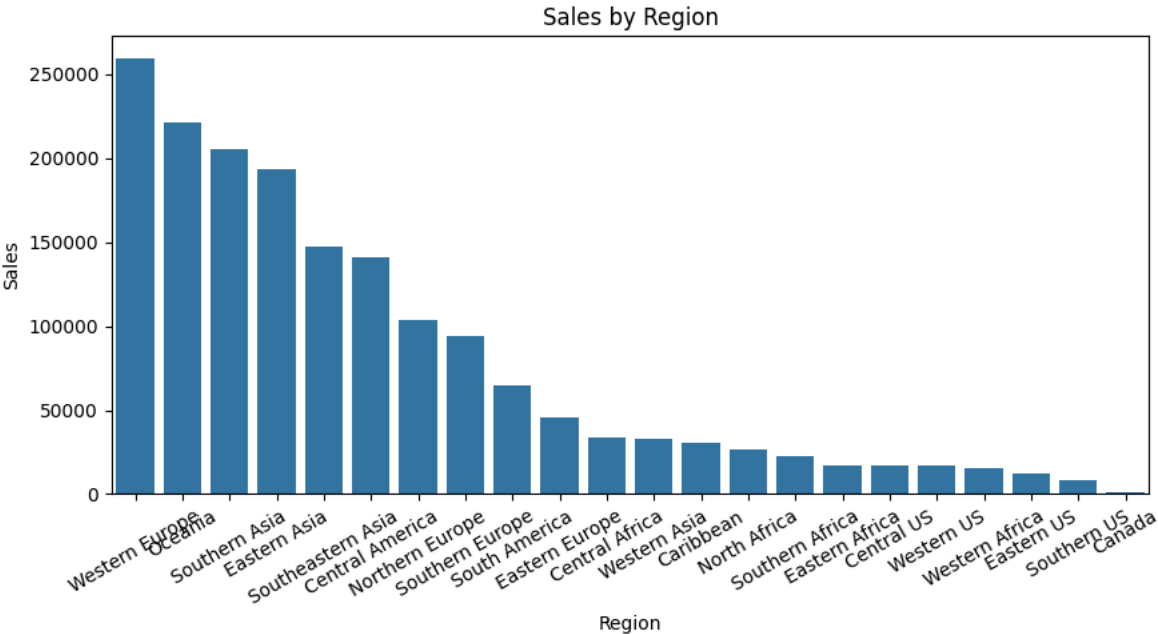
```
Category
Technology      145068.68
Furniture        82188.95
Office Supplies   61662.81
Name: Profit, dtype: float64
```



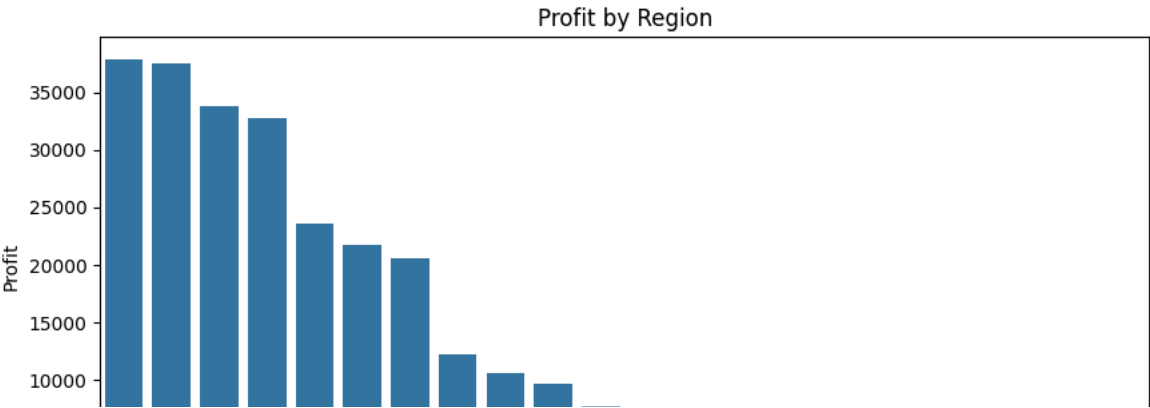
Sales by Region:

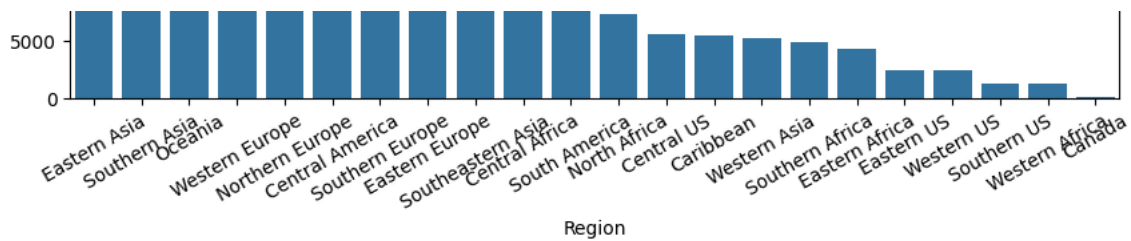
```
Region
Western Europe      259576.28
Oceania              220809.08
Southern Asia        205466.26
Eastern Asia         193590.60
Southeastern Asia    147337.60
Central America      140966.21
Northern Europe       103888.63
Southern Europe        94441.11
South America         64550.54
Eastern Europe         45662.19
```

```
Central Africa      33810.48
Western Asia       33308.21
Caribbean         30746.74
North Africa       26266.44
Southern Africa    22262.28
Eastern Africa     17387.94
Central US         17187.94
Western US         16729.95
Western Africa     15478.79
Eastern US         12256.44
Southern US        8360.75
Canada            887.01
Name: Sales, dtype: float64
```



```
Profit by Region:
Region
Eastern Asia      37911.16
Southern Asia     37536.60
Oceania          33777.56
Western Europe    32783.40
Northern Europe   23552.73
Central America   21741.95
Southern Europe   20557.50
Eastern Europe    12228.45
Southeastern Asia 10561.80
Central Africa    9668.22
South America     7723.58
North Africa     7416.42
Central US        5676.02
Caribbean        5463.82
Western Asia     5262.35
Southern Africa   4876.32
Eastern Africa    4302.72
Eastern US        2544.89
Western US        2458.34
Southern US       1384.72
Western Africa    1278.17
Canada           213.72
Name: Profit, dtype: float64
```

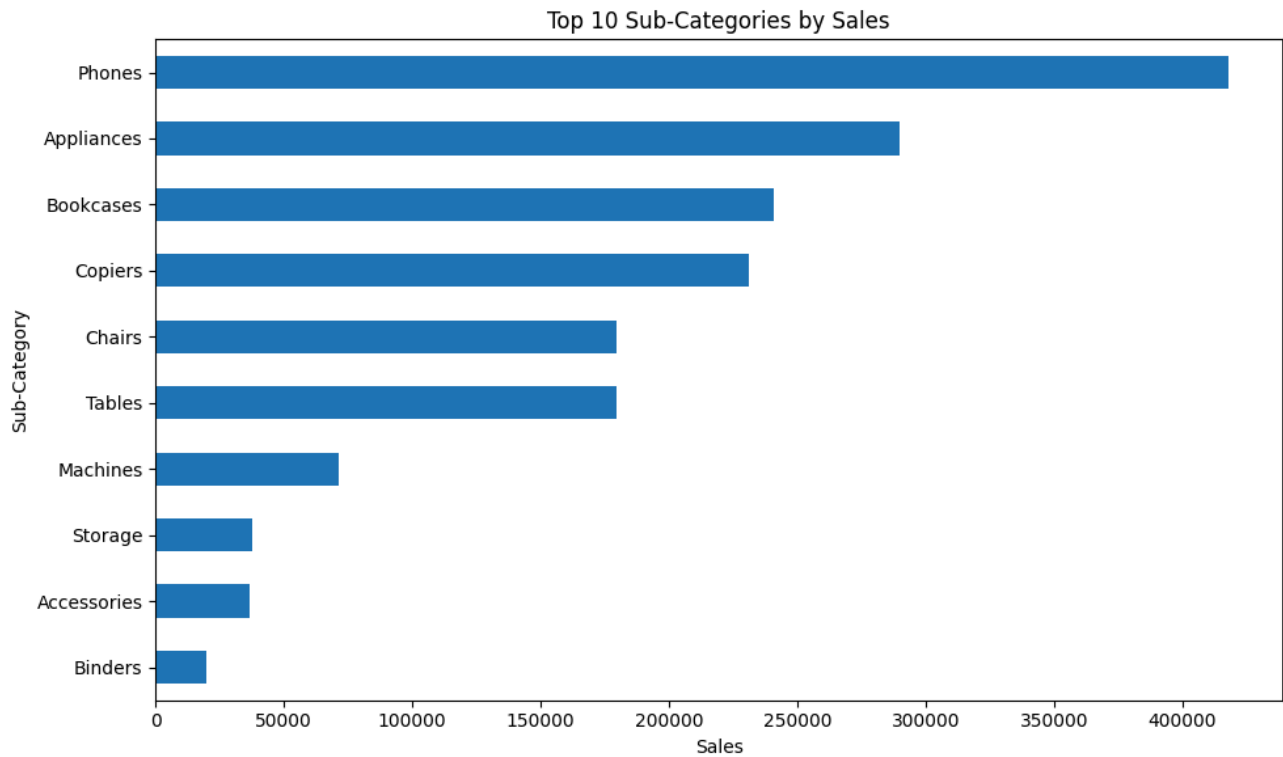




Top Sub-Categories by Sales:

Sub-Category	Sales
Phones	417939.31
Appliances	289711.30
Bookcases	240688.73
Copiers	231104.67
Chairs	179648.44
Tables	179292.88
Machines	71366.87
Storage	37592.32
Accessories	36572.17
Binders	19751.19

Name: Sales, dtype: float64



=====

CORRELATION MATRIX

=====

i\jRow ID	Postal Code	Sales	Quantity	Discount	Profit	Shipping Cost
i\jRow ID	1.000000	-0.065876	-0.220878	-0.173947	0.032788	-0.043650
Postal Code	-0.065876	1.000000	0.090598	0.051529	0.003416	0.019801
Sales	-0.220878	0.090598	1.000000	0.555691	-0.187997	0.533676
Quantity	-0.173947	0.051529	0.555691	1.000000	0.034045	0.154481
Discount	0.032788	0.003416	-0.187997	0.034045	1.000000	-0.505190
Profit	-0.043650	0.019801	0.533676	0.154481	-0.505190	1.000000
Shipping Cost	-0.284645	0.117999	0.644891	0.356324	-0.169216	0.290593

Shipping Cost	-0.284645
Postal Code	0.117999
Sales	0.644891
Quantity	0.356324
Discount	-0.169216
Profit	0.290593
Shipping Cost	1.000000

